Node.js Interview Cheat Sheet

Node.js Interview Cheat Sheet (2 YOE)

Frequently Asked Questions & Answers — tailored for Express, REST APIs, Microservices, AWS Lambda & Oracle DB

1. What is Node.js and why is it used?

Q: What is Node.js? Why use it for backend development?

Node.js is a JavaScript runtime built on Chrome's V8 engine that lets JavaScript run outside the browser, on a server. It uses an **event-driven, non-blocking I/O model**, making it lightweight and efficient for I/O-heavy applications (APIs, real-time apps, microservices).

Why teams choose it: one language (JS) across frontend and backend, huge npm ecosystem, excellent for handling many concurrent I/O operations (API calls, DB queries) without spawning a thread per request.

```
// A minimal Node.js HTTP server — no framework needed
const http = require("http");
const server = http.createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "text/plain" });
  res.end("Hello from Node.js");
});
server.listen(3000, () => console.log("Server running on port 3000"));
```

2. Is Node.js single-threaded? How does it handle concurrency?

Node.js runs your JavaScript on a **single main thread**, but I/O operations (file reads, DB queries, network calls) are delegated to the **libuv** thread pool (or the OS kernel for things like networking) and don't block that main thread. When an operation completes, its callback is queued and picked up by the **event loop**.

```
Your JS code (single thread)
        |
        ▼
  Async operation requested (fetch, fs.readFile, DB query)
        |
        ▼
  Delegated to libuv thread pool / OS kernel
        |
        ▼
  Main thread keeps handling OTHER requests
        |
        ▼
  Operation completes → callback queued
        |
        ▼
  Event loop picks it up when the call stack is empty
```

This is why Node.js handles thousands of concurrent connections efficiently on one thread — it's built for I/O concurrency, not CPU parallelism. CPU-heavy work (image processing, heavy computation) should be offloaded to **worker threads** or a separate service, since it *would* block the event loop.

3. Node.js Event Loop Phases (detailed)

Beyond the simplified "microtask vs macrotask" model, Node's event loop (via libuv) has distinct phases each tick:

Phase	Handles
Timers	`setTimeout`, `setInterval` callbacks whose time has elapsed
Pending callbacks	Some system-level callbacks (e.g. TCP errors) deferred from the previous cycle
Idle, prepare	Internal use only
Poll	Retrieves new I/O events; executes I/O callbacks (file reads, DB responses)
Check	`setImmediate()` callbacks
Close callbacks	e.g. `socket.on('close', ...)`

`process.nextTick()` and resolved Promise callbacks (microtasks) run **between every phase**, before the loop moves on — they have the highest priority.

```
console.log("start");
setTimeout(() => console.log("timeout"), 0);   // Timers phase
setImmediate(() => console.log("immediate"));  // Check phase
process.nextTick(() => console.log("nextTick")); // Runs before next phase
Promise.resolve().then(() => console.log("promise"));
console.log("end");

// Output: start, end, nextTick, promise, timeout, immediate
// (order of timeout vs immediate can vary depending on context)
```

4. Blocking vs Non-Blocking Code

```
const fs = require("fs");

// Blocking (synchronous) — halts the entire thread until the file is read
const data = fs.readFileSync("file.txt", "utf-8");
console.log(data);
console.log("This runs AFTER the file is read");

// Non-blocking (asynchronous) — thread continues immediately
fs.readFile("file.txt", "utf-8", (err, data) => {
  console.log(data);
});
console.log("This runs BEFORE the file finishes reading");
```

Rule of thumb: always prefer the async (`readFile`, `*Sync` avoided) versions in a server — a single blocking call freezes the event loop for **every** concurrent request, not just the one that triggered it.

5. Modules: require vs import

Node traditionally uses **CommonJS** (`require` / `module.exports`); modern Node (with `"type": "module"` in `package.json` or `.mjs` files) also supports **ES Modules** (`import` / `export`).

```
// CommonJS
// math.js
function add(a, b) { return a + b; }
module.exports = { add };
// app.js
const { add } = require("./math");

// ES Modules
// math.mjs
export function add(a, b) { return a + b; }
// app.mjs
import { add } from "./math.mjs";
```

Key practical difference: `require` is synchronous and can be called conditionally/anywhere; `import` is static and hoisted, analyzed before execution.

6. package.json vs package-lock.json

	`package.json`	`package-lock.json`
Purpose	Declares direct dependencies (with version **ranges**, e.g. `^4.18.0`)	Locks the **exact** resolved version of every dependency (including nested/transitive ones)
Committed to git?	Yes	Yes — ensures identical installs across machines/CI
Edited manually?	Sometimes (adding a new dep)	Rarely — auto-generated by npm

```
npm install express        # adds to package.json + package-lock.json
npm ci                     # installs EXACTLY what's in package-lock.json — used in CI/CD for reproducibility
```

---

7. Express.js Basics

Express is the most common Node web framework — it simplifies routing, middleware, and request/response handling on top of the raw `http` module.

```
const express = require("express");
const app = express();

app.use(express.json()); // parses JSON request bodies

app.get("/users/:id", (req, res) => {
  res.json({ id: req.params.id, name: "Sapna" });
});

app.post("/users", (req, res) => {
  const { name } = req.body;
  res.status(201).json({ message: `User ${name} created` });
});

app.listen(3000, () => console.log("Server started"));
```

---

8. What is Middleware in Express?

A middleware function has access to `req`, `res`, and `next()`, and can inspect/modify the request-response cycle, end it, or pass control to the next handler.

```
function logger(req, res, next) {
  console.log(`${req.method} ${req.url}`);
  next(); // pass control onward — forgetting this hangs the request!
}
app.use(logger);
```

Types of middleware | Type | Example | |—|—| | Application-level | `app.use()`, runs for every matching request | | Router-level | Attached to an `express.Router()` instance | | Built-in | `express.json()`, `express.static()` | | Third-party | `cors`, `helmet`, `morgan` | | Error-handling | Takes 4 args `(err, req, res, next)`, defined last |

Error-handling middleware:

```
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ message: "Something went wrong" });
});
```

Async error handling — Express doesn't catch rejected promises from `async` route handlers by default, so wrap them:

```
const asyncHandler = (fn) => (req, res, next) => fn(req, res, next).catch(next);

app.get("/users/:id", asyncHandler(async (req, res) => {
  const user = await getUserFromDb(req.params.id);
  if (!user) {
    const err = new Error("User not found");
    err.status = 404;
    throw err;
  }
  res.json(user);
}));
```

9. REST API Design & HTTP Status Codes

Method	Purpose	Idempotent?
`GET`	Retrieve a resource	Yes
`POST`	Create a new resource	No
`PUT`	Replace a resource entirely	Yes
`PATCH`	Partially update a resource	No (in practice, often treated as yes)
`DELETE`	Remove a resource	Yes

Status Code	Meaning
`200 OK`	Success (GET, PUT, PATCH)
`201 Created`	Resource created (POST)
`204 No Content`	Success, nothing to return (DELETE)
`400 Bad Request`	Invalid client input
`401 Unauthorized`	Missing/invalid authentication
`403 Forbidden`	Authenticated but not permitted
`404 Not Found`	Resource doesn't exist
`409 Conflict`	Duplicate/conflicting state
`422 Unprocessable Entity`	Validation failed
`500 Internal Server Error`	Unhandled server-side error

REST design principles: resources as nouns in URLs (`/users/:id` , not `/getUser`), use HTTP methods to express the action, stateless requests (no server-side session dependency between calls), consistent error response shape, versioning (`/api/v1/...`).

10. Server-Side Pagination

Since large datasets shouldn't be sent in one response, paginate on the backend.

```
app.get("/customers", async (req, res) => {
  const page = parseInt(req.query.page) || 1;
  const limit = parseInt(req.query.limit) || 20;
  const offset = (page - 1) * limit;

  const customers = await db.query(
    `SELECT * FROM customers ORDER BY id OFFSET :offset ROWS FETCH NEXT :limit ROWS ONLY`,
    { offset, limit }
  );
  const total = await db.query(`SELECT COUNT(*) as count FROM customers`);

  res.json({
    data: customers,
    page,
    totalPages: Math.ceil(total[0].count / limit)
  });
});
```

Offset-based (`OFFSET/FETCH`)	Cursor-based (`WHERE id > :lastId`)
Simple, supports jumping to any page	No “page N” jump, but stable under inserts/deletes
Can skip/duplicate rows if data changes between requests	More consistent for real-time/frequently-changing data
Good for admin dashboards, reports	Good for infinite scroll, feeds

11. SQL: Oracle DB, Connection Pooling & CTEs

Connection pooling — opening a new DB connection per request is expensive; a pool maintains a set of reusable connections.

```
const oracledb = require("oracledb");

async function initPool() {
  await oracledb.createPool({
    user: process.env.DB_USER,
    password: process.env.DB_PASSWORD,
    connectString: process.env.DB_CONNECT_STRING,
    poolMin: 2,
    poolMax: 10
  });
}

async function getCustomer(id) {
  const connection = await oracledb.getConnection(); // borrowed from the pool
  try {
    const result = await connection.execute(
      `SELECT * FROM customers WHERE id = :id`,
      [id]
    );
    return result.rows;
  } finally {
    await connection.close(); // returns it to the pool, doesn't destroy it
  }
}
```

CTEs (Common Table Expressions) — a named temporary result set defined with `WITH`, used to break complex queries into readable steps and enable recursion.

```
-- CTE example: customers with more than 5 orders
WITH order_counts AS (
  SELECT customer_id, COUNT(*) AS total_orders
  FROM orders
  GROUP BY customer_id
)
SELECT c.name, oc.total_orders
FROM customers c
JOIN order_counts oc ON c.id = oc.customer_id
WHERE oc.total_orders > 5;
```

Benefits over a subquery: more readable, can be referenced multiple times in the same query, and supports recursive CTEs

(e.g. traversing an org hierarchy).

Parameterized queries (as shown above with `:id`) are essential to prevent **SQL injection** — never concatenate raw user input into a query string.

---

12. ORMs — Why and When

An ORM (Sequelize, TypeORM, Prisma) maps database tables to JS objects/classes, so you write JS instead of raw SQL for common operations, and get built-in protection against SQL injection, migrations, and model validation.

```
// Sequelize example
const Customer = sequelize.define("Customer", {
  name: DataTypes.STRING,
  email: DataTypes.STRING
});

const customer = await Customer.findByPk(1);
const newCustomer = await Customer.create({ name: "Sapna", email: "s@x.com" });
```

Trade-off: ORMs speed up common CRUD and reduce boilerplate, but complex reporting queries (heavy joins, CTEs, window functions) are often still easier and more performant as raw SQL — many teams (like ones using Oracle heavily) mix both: ORM for standard entities, raw parameterized SQL for reporting/analytics.

---

13. Authentication in Node/Express (JWT)

```
const jwt = require("jsonwebtoken");

// On login
app.post("/login", async (req, res) => {
  const user = await validateCredentials(req.body);
  const token = jwt.sign(
    { userId: user.id, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: "1h" }
  );
  res.json({ token });
});

// Auth middleware — protects downstream routes
function authenticate(req, res, next) {
  const authHeader = req.headers.authorization; // "Bearer <token>"
  const token = authHeader?.split(" ")[1];
  if (!token) return res.status(401).json({ message: "No token provided" });

  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (err) {
    res.status(401).json({ message: "Invalid or expired token" });
  }
}

app.get("/profile", authenticate, (req, res) => {
  res.json({ userId: req.user.userId });
});
```

Store secrets (`JWT_SECRET`, DB creds) in environment variables (`.env` + `dotenv`), never hard-coded in source. Prefer short-lived access tokens plus a refresh-token flow for production systems.

---

14. Security Best Practices

Concern	Mitigation
Common HTTP header vulnerabilities	`helmet` middleware sets secure headers by default
Cross-Origin requests	`cors` middleware, configured to allow only trusted origins
SQL injection	Parameterized queries / ORM (never string-concatenate SQL)
Brute-force / abuse	Rate limiting (`express-rate-limit`)
Sensitive data in code	Environment variables (`dotenv`), never commit secrets
Unvalidated input	Validation libraries (`Joi` , `express-validator`) at the API boundary
Outdated dependencies	`npm audit` , keep packages patched

```
const helmet = require("helmet");
const cors = require("cors");
const rateLimit = require("express-rate-limit");

app.use(helmet());
app.use(cors({ origin: "https://trusted-frontend.com" }));
app.use(rateLimit({ windowMs: 15 * 60 * 1000, max: 100 })); // 100 req / 15 min per IP
```

15. Streams & Buffers

A **Buffer** is Node's way of handling raw binary data. A **Stream** processes data in chunks instead of loading it all into memory at once — critical for large files or large API responses.

```
const fs = require("fs");

// Without streams — loads the ENTIRE file into memory
const data = fs.readFileSync("huge-file.csv");

// With streams — processes chunk by chunk, low memory footprint
const readStream = fs.createReadStream("huge-file.csv");
const writeStream = fs.createWriteStream("output.csv");
readStream.pipe(writeStream); // pipes chunks directly, handles backpressure automatically
```

Types: Readable (source, e.g. `fs.createReadStream`), Writable (destination, e.g. HTTP response), Duplex (both, e.g. TCP sockets), Transform (modifies data as it passes through, e.g. compression).

Relevant for the "automated bulk upload" type of workflow — streaming a large CSV upload instead of buffering it all avoids memory spikes.

16. Scaling Node.js: Cluster Module & Worker Threads

Since Node runs on a single thread by default, a single process can't use more than one CPU core. Two ways to scale:

`cluster` **module** — forks multiple worker processes (one per CPU core), each running its own event loop, sharing the same port.

```
const cluster = require("cluster");
const os = require("os");

if (cluster.isPrimary) {
  os.cpus().forEach(() => cluster.fork());
} else {
  require("./app"); // each worker runs the actual Express app
}
```

Worker threads (`worker_threads` module) — for CPU-intensive tasks (image processing, heavy computation) within a single process, without blocking the main event loop.

In production, process managers like **PM2** typically handle clustering, auto-restarts, and zero-downtime reloads instead of hand-rolling the `cluster` module.

17. Microservices Architecture

Splits an application into small, independently deployable services, each owning a specific business capability (e.g. a “Customer Service”, “Allocation Service”), communicating over the network (usually REST or messaging).

Key concepts - **API Gateway** — single entry point that routes requests to the right microservice, and can handle cross-cutting concerns (auth, rate limiting, logging) in one place instead of duplicating them per service. - **Service-to-service communication** — synchronous (REST/HTTP, gRPC) or asynchronous (message queues like SQS/RabbitMQ) depending on whether the caller needs an immediate response. - **Independent deployability** — each service has its own codebase, CI/CD pipeline, and can scale independently.

```
Client → API Gateway → routes to →
   ├── Customer Service    (Node.js + Oracle DB)
   ├── Allocation Service  (Node.js + AWS Lambda)
   └── Reporting Service   (Node.js + Oracle DB)
```

Trade-offs vs a monolith: better scalability and independent deployment, but adds operational complexity (distributed tracing, network failures between services, data consistency across services).

---

18. AWS Lambda & Serverless Basics

Lambda runs your Node.js function code without you managing a server — it’s invoked in response to an event (an API Gateway request, an S3 upload, a queue message) and scales automatically.

```
// A simple Lambda handler
exports.handler = async (event) => {
  const { id } = event.pathParameters;
  const customer = await getCustomerFromDb(id);
  return {
    statusCode: 200,
    body: JSON.stringify(customer)
  };
};
```

API Gateway + Lambda is a common serverless REST pattern: API Gateway receives the HTTP request, maps it to a Lambda invocation, and returns the Lambda’s response to the client.

Cold starts — when a Lambda hasn’t run recently, AWS has to initialize a new execution environment before running your code, adding latency to that first request. Mitigations: keep the deployment package small, minimize heavy imports at the top of the file, use **provisioned concurrency** for latency-sensitive endpoints.

Lambda vs a long-running Express server: | | Lambda | Express server (EC2/container) | |—|—|—| | Scaling | Automatic, per-request | Manual/auto-scaling groups | | Billing | Pay per invocation/duration | Pay for uptime regardless of traffic | | Cold starts | Yes | No (always warm) | | Best for | Bursty/unpredictable traffic, event-driven tasks | Steady traffic, long-lived connections (WebSockets) |

---

19. CI/CD for Node.js Applications

A typical pipeline: **lint → test → build → deploy**, run automatically on each push/PR.

```
# Simplified example (GitHub Actions style)
steps:
  - run: npm ci                # reproducible install from package-lock.json
  - run: npm run lint
  - run: npm test              # unit + BDD tests
  - run: npm run build
  - run: npm run deploy        # e.g. deploy Lambda package, or push a Docker image
```

Why `npm ci` over `npm install` in CI: it installs the **exact** locked versions and fails if `package-lock.json` is out of sync — guarantees the build is reproducible across environments.

---

20. Testing: Unit Tests vs BDD (Cucumber.js)

Unit/integration tests (Jest/Mocha) test individual functions or routes in isolation.

```
test("GET /users/:id returns a user", async () => {
  const res = await request(app).get("/users/1");
  expect(res.status).toBe(200);
  expect(res.body.name).toBe("Sapna");
});
```

BDD with Cucumber.js — describes behavior in plain-language `Gherkin` syntax (`Given/When/Then`), readable by non-developers (QA, product), then maps each line to actual test code via step definitions.

```
# customer.feature
Feature: Customer retrieval
  Scenario: Fetch an existing customer
    Given a customer with id "123" exists
    When I request the customer by id "123"
    Then the response status should be 200
    And the response should contain the customer's name
```

```
// step definitions — customer.steps.js
const { Given, When, Then } = require("@cucumber/cucumber");
const assert = require("assert");

Given("a customer with id {string} exists", async function (id) {
  this.customerId = id;
  await seedTestCustomer(id);
});

When("I request the customer by id {string}", async function (id) {
  this.response = await request(app).get(`/customers/${id}`);
});

Then("the response status should be {int}", function (statusCode) {
  assert.strictEqual(this.response.status, statusCode);
});
```

Why BDD matters in enterprise teams: it creates a shared, readable spec between business/QA and engineering, and doubles as living documentation of expected behavior.

---

21. Logging & Monitoring

Use a structured logger instead of scattered `console.log` calls, so logs can be filtered, leveled (`info` / `warn` / `error`), and shipped to a monitoring system.

```
const winston = require("winston");
const logger = winston.createLogger({
  level: "info",
  format: winston.format.json(),
  transports: [new winston.transports.Console(), new winston.transports.File({ filename: "app.log" })]
});

logger.info("Server started on port 3000");
logger.error("Failed to connect to Oracle DB", { error: err.message });
```

`morgan` is commonly layered on top of Express specifically for HTTP request logging (`method`, `url`, `status`, `response time`).

---

22. Handling Uncaught Errors at the Process Level

```
process.on("uncaughtException", (err) => {
  logger.error("Uncaught exception:", err);
  process.exit(1); // crash intentionally — the process is in an unknown state
});

process.on("unhandledRejection", (reason) => {
  logger.error("Unhandled promise rejection:", reason);
});
```

Why exit rather than “recover”: after an uncaught exception, the process may be in a corrupted state (partially released

resources, inconsistent in-memory state). Best practice is to log it, exit cleanly, and let a process manager (PM2, Kubernetes, Lambda's own retry) restart a fresh instance — not to keep limping along.

---

23. Environment Variables & Configuration

```
require("dotenv").config();

const config = {
  port: process.env.PORT || 3000,
  dbConnectString: process.env.DB_CONNECT_STRING,
  jwtSecret: process.env.JWT_SECRET
};
```

`.env` files hold local secrets/config and should be **git-ignored**; actual values in production come from the deployment platform's secret store (AWS Secrets Manager/Parameter Store, CI/CD secrets) — never committed to the repo.

---

24. Common Node.js Interview One-Liners

Q: What is the difference between `process.nextTick()` and `setImmediate()`? `process.nextTick()` runs its callback **before** the event loop continues to the next phase (highest priority, can starve I/O if overused). `setImmediate()` runs in the **check** phase, after I/O callbacks in the current loop iteration.

Q: What is `npm` vs `npx`? `npm` installs and manages packages. `npx` executes a package's binary directly — either one that's already installed locally, or by fetching it temporarily — without needing a global install (e.g. `npx create-react-app`).

Q: What is the difference between `dependencies` and `devDependencies`? `dependencies` are required at runtime in production (Express, jsonwebtoken). `devDependencies` are only needed during development/testing (Jest, ESLint, nodemon) and aren't installed in a production build with `npm install --production`.

Q: How do you handle a slow/failing downstream API call in a microservice? Set a request timeout, add retries with backoff for transient failures, and consider a circuit breaker pattern so repeated failures don't keep hammering a downstream service that's already struggling.

Q: What is the purpose of a `.env` file, and why not commit it? It holds environment-specific config/secrets outside the codebase, so the same code can run in dev/staging/prod with different values, and no credentials leak into version control.

Q: How does Node.js differ from a traditional multi-threaded server (e.g. Java/Tomcat)? Traditional servers typically spawn a thread per request, which costs memory per connection but parallelizes CPU work naturally. Node uses a single-threaded event loop with non-blocking I/O — very memory-efficient for high-concurrency I/O-bound workloads, but CPU-heavy tasks need to be explicitly offloaded (worker threads, separate services) to avoid blocking everyone.

Quick Reference — What to Reach For

Problem	Solution
Large file/response handling without high memory use	Streams
Repeated DB connection overhead	Connection pooling
Complex multi-step SQL logic	CTEs (`WITH`)
Preventing SQL injection	Parameterized queries / ORM
Protecting routes	JWT + auth middleware
Cross-origin requests	`cors`
Common HTTP security headers	`helmet`
Abuse/brute-force protection	`express-rate-limit`
CPU-heavy work blocking the event loop	Worker threads / separate service
Using all CPU cores	`cluster` module or PM2
Async route errors not being caught	Wrap handlers (`asyncHandler`) or use error-handling middleware
Reproducible installs in CI	`npm ci` with `package-lock.json`
Shared, readable test specs with QA	Cucumber.js (BDD)
Bursty/serverless workloads	AWS Lambda + API Gateway
Centralized cross-cutting concerns across services	API Gateway